Robot de auto-balanceo de dos ruedas con aprendizaje por refuerzo profundo

Informe final — CC3104 Aprendizaje por Refuerzo

Christian Echeverría, 221441
Mario Betancourt, 23440
Universidad del Valle de Guatemala

Repositorio: `https://github.com/Ctribsz/ProyectoFinal-APR`

Video: `https://youtu.be/TeNxtZMx2g0`

9 de noviembre de 2026

Resumen

Se comparan cuatro algoritmos de control continuo de Stable-Baselines3 (DDPG, TD3, SAC y PPO) en la tarea de balancear un robot de dos ruedas simulado en MuJoCo, y se contrastan con un controlador PID en cascada. Los cuatro algoritmos se afinaron con el mismo presupuesto (40 pruebas de Optuna de 200 000 pasos) y se entrenaron con un millón de pasos en un entorno con comandos de velocidad y empujones, con 10 semillas cada uno. Después se evaluaron en cuatro niveles de dificultad, en una prueba de empujón y entrenando con aleatorización de dominio. TD3 obtiene el mejor desempeño en las condiciones de entrenamiento (IQM del retorno 891 de 1000) y se recupera más rápido de los empujones. SAC es el más eficiente en muestras y el más consistente entre semillas. PPO es el que mejor generaliza, sin haberlo entrenado ahí, al nivel con ruido de sensores, latencia y física aleatorizada. Los diagnósticos internos confirman el mecanismo que distingue a TD3 de DDPG: DDPG sobreestima el valor Q en unos 41 puntos de retorno, mientras que el doble crítico de TD3 lo subestima. Todos los agentes superan al PID cuando hay ruido y latencia, aunque el PID tolera empujones aislados más fuertes.

Índice

1. Descripción del problema **2**

2. Análisis **3**
2.1. Qué necesita un buen controlador 3
2.2. Por qué estos algoritmos 3
2.3. Qué hace falta para que la comparación sea justa 3
2.4. Hipótesis 3

3. Propuesta de solución **4**

4. Descripción de la solución **4**
4.1. Modelo físico 4
4.2. Observación, acción y recompensa 4
4.3. Niveles de dificultad 5
4.4. Línea base: PID en cascada 5
4.5. Pipeline común 5
4.6. Afinamiento 6
4.7. Protocolo experimental 6

5. Herramientas aplicadas **6**
5.1. MuJoCo . . . 6
5.2. Gymnasium . . . 7
5.3. Stable-Baselines3 . . . 7
5.4. Optuna . . . 7
5.5. rliable . . . 7
5.6. Otras herramientas . . . 8
5.7. Herramienta de IA generativa . . . 8
5.7.1. Etapa 1: definición del proyecto . . . 8
5.7.2. Etapa 2: implementación . . . 9
5.7.3. Evolución de la idea . . . 11
5.7.4. Entendimiento de la solución . . . 12

6. Resultados **13**
6.1. Métricas . . . 13
6.2. E1 y E2: eficiencia de muestras y de tiempo . . . 13
6.3. E3 y E6: generalización a los niveles 0–3 y comparación con el PID . . . 14
6.4. E4: prueba de empujón . . . 15
6.5. E5: entrenar con aleatorización de dominio . . . 15
6.6. Diagnósticos internos . . . 16
6.7. Contraste de hipótesis . . . 16

7. Conclusión **17**

1. Descripción del problema

Un robot de dos ruedas con auto-balanceo es un péndulo invertido montado sobre ruedas: el chasis se cae si nadie lo corrige, y la única forma de mantenerlo vertical es mover las ruedas por debajo de su centro de masa. Es un problema clásico de control porque combina tres dificultades:

- **Es inestable en lazo abierto.** Sin una acción de control en cada instante, cualquier inclinación crece hasta que el robot cae.
- **Tiene fase no mínima en velocidad.** Para avanzar o frenar, el robot primero tiene que mover las ruedas en el sentido contrario e inclinarse; un controlador que persigue la velocidad de forma directa lo desestabiliza.
- **Debe funcionar con información imperfecta.** Un robot real solo conoce su estado a través de una IMU y de encoders en las ruedas, con ruido, saturación y latencia, y recibe perturbaciones externas (empujones, cambios de masa o de fricción).

El aprendizaje por refuerzo [15] profundo ofrece varios algoritmos para control continuo que pueden aprender una política directamente a partir de la interacción, sin un modelo del sistema. Pero cada uno tiene un mecanismo distinto (crítico determinista, doble crítico, máxima entropía, gradiente de política con recorte), y en la literatura es común compararlos con valores por defecto y una sola semilla, lo que no permite saber cuál es mejor ni por qué [10, 1].

El proyecto responde la siguiente pregunta:

> ¿Qué algoritmo ofrece el mejor desempeño para el auto-balanceo robusto ante perturbaciones, y qué mecanismos técnicos de cada algoritmo explican ese resultado?

2. Análisis

2.1. Qué necesita un buen controlador

Del problema se desprenden los requisitos que debe cumplir una política:

1. Mantener la vertical, es decir, una inclinación θ cercana a cero, durante todo el episodio.
2. Seguir comandos de velocidad lineal v^* y de giro $\dot{\psi}^*$ cuando se le piden.
3. Recuperarse de empujones, en el menor tiempo posible y tolerando la mayor fuerza posible.
4. Hacerlo con acciones suaves y poco esfuerzo, porque un motor real no soporta cambios bruscos de torque y las acciones de alta frecuencia amplifican el ruido de los sensores.
5. Usar solo lo que mediría un robot real, para que la política sea, en principio, transferible.

2.2. Por qué estos algoritmos

Los cuatro algoritmos son los métodos estándar de control continuo y representan mecanismos distintos que se pueden contrastar:

- **DDPG** [11]: actor-crítico determinista *off-policy*. Aprende un crítico $Q(s, a)$ y un actor que lo maximiza. Es conocido por sobreestimar Q y ser inestable.
- **TD3** [7]: corrige a DDPG con tres cambios: el mínimo de dos críticos (en la línea de *double Q-learning* [19]), retrasar la actualización del actor y suavizar la acción objetivo con ruido.
- **SAC** [8, 9]: *off-policy* con política estocástica, que maximiza el retorno más la entropía de la política. También usa dos críticos y ajusta de forma automática el peso de la entropía.
- **PPO** [14]: *on-policy*, de gradiente de política con un objetivo recortado que limita cuánto cambia la política en cada actualización. Estima ventajas con GAE [13].

Como línea base se usa un controlador clásico: si el aprendizaje por refuerzo no supera a un PID bien ajustado, no aporta nada en este problema [3].

2.3. Qué hace falta para que la comparación sea justa

Henderson et al. [10] muestran que, en RL profundo, la semilla y los hiperparámetros pueden cambiar más el resultado que el algoritmo. Por eso la comparación necesita:

- el mismo entorno, la misma red y el mismo código de entrenamiento para los cuatro;
- el mismo presupuesto de afinamiento y de pasos de entorno;
- varias semillas y una estadística robusta entre ellas, como la media intercuartil (IQM) con intervalos de confianza por bootstrap [1];
- diagnósticos internos que expliquen el resultado, no solo el retorno;
- escenarios de evaluación más duros que los de entrenamiento, para medir generalización.

2.4. Hipótesis

Del análisis de los mecanismos salen cuatro hipótesis, escritas antes de los experimentos finales:

H1 TD3 sobreestima menos Q que DDPG y por eso entrena de forma más estable.

H2 SAC se recupera mejor de empujones grandes, por la exploración que le da la entropía.

H3 PPO necesita más pasos de entorno, pero tiene menor varianza entre semillas.

H4 Los agentes igualan al PID en el nivel 0 y lo superan en los niveles 2 y 3.

3. Propuesta de solución

La propuesta tiene cinco piezas:

1. **Un entorno propio en MuJoCo**, con la interfaz de Gymnasium, en el que el robot solo observa lo que medirían una IMU y los encoders, y con **cuatro niveles de dificultad** que comparten el mismo espacio de observación. Así un agente entrenado en un nivel se puede evaluar en todos.
2. **Un PID en cascada** como línea base, evaluado con el mismo código que los agentes.
3. **Un único pipeline** de entrenamiento, afinamiento y evaluación para los cuatro algoritmos, sobre Stable-Baselines3, en el que cada algoritmo se distingue solo por su archivo de configuración.
4. **Un protocolo experimental** con el mismo presupuesto para todos: afinamiento con Optuna, corridas finales con varias semillas, evaluación con semillas fijas y diagnósticos internos.
5. **Seis experimentos** que responden partes distintas de la pregunta (tabla 1).

Tabla 1: Experimentos.

Id	Experimento	Pregunta
E1	Curvas por pasos de entorno	¿Quién aprende con menos interacción?
E2	Curvas por tiempo de reloj	¿Quién aprende más rápido en la práctica?
E3	Desempeño final en los niveles 0–3	¿Quién generaliza a condiciones más duras?
E4	Prueba de empujón	¿Cuánta fuerza tolera cada uno y qué tan rápido se recupera?
E5	Entrenar con aleatorización (nivel 3)	¿La aleatorización mejora la robustez?
E6	Comparación contra el PID	¿Dónde aporta el RL frente al control clásico?

4. Descripción de la solución

4.1. Modelo físico

El robot se modela en MJCF como un chasis de 1.0 kg y 0.30 m de alto que pivota sobre dos ruedas de 4 cm de radio separadas 18 cm, con un motor de hasta 0.3 N m por rueda. La física corre a 500 Hz y el control a 100 Hz. Los sensores imitan a un robot real: una IMU (orientación, giroscopio con saturación en $\pm 2000\,^\circ/\mathrm{s}$ y acelerómetro) y encoders en las ruedas.

4.2. Observación, acción y recompensa

La observación tiene nueve valores y solo usa lo que mediría un robot real (tabla 2); nunca la posición absoluta ni el estado interno del simulador.

Tabla 2: Espacio de observación.

Índice	Variable	Fuente equivalente
0	Pitch θ	IMU
1–2	Velocidad de pitch $\dot{\theta}$ y de giro $\dot{\psi}$	Giroscopio
3–4	Velocidad de cada rueda	Encoders
5–6	Comandos v^* y $\dot{\psi}^*$	Comando externo
7–8	Acción anterior	Memoria del controlador

La acción es un comando normalizado $a \in [-1, 1]^2$, uno por motor. La recompensa por paso es una suma ponderada de componentes, que se registran por separado:

$$r_t = w_{\text{alive}} - w_\theta\,\theta^2 - w_{\dot\theta}\,\dot\theta^2 - w_v(v - v^*)^2 - w_{\dot\psi}(\dot\psi - \dot\psi^*)^2 - w_{\text{effort}}\|a_t\|^2 - w_{\text{smooth}}\|a_t - a_{t-1}\|^2. \quad (1)$$

El episodio termina si $|\theta| > 0.6\,\text{rad}$ (el robot cayó) y se trunca a los 1000 pasos (10 s). El retorno máximo es 1000.

4.3. Niveles de dificultad

Tabla 3: Niveles de dificultad. Todos comparten el espacio de observación: se entrena en uno y se evalúa en todos.

Nivel	Qué agrega
0	Balanceo en reposo.
1	Empujones aleatorios de hasta 8 N durante 0.1 s, uno cada 2 s en promedio, en la parte alta del chasis.
2	Comandos de velocidad (± 0.5 m/s) y de giro (± 1 rad/s) que cambian cada 2–4 s. **Nivel de entrenamiento.**
3	Ruido gaussiano en los sensores, latencia de 0–20 ms, masa ± 20 %, fricción ± 30 % y torque ± 10 % [16]. La recompensa usa las lecturas limpias.

4.4. Línea base: PID en cascada

Un lazo externo convierte el error de velocidad en un pitch objetivo, un lazo interno convierte el error de pitch en torque, y un término diferencial entre ruedas controla el giro. El PID usa solo la observación, igual que los agentes, y se evalúa con el mismo código. Sus ganancias se eligieron con una búsqueda en rejilla en el nivel 0. El lazo de velocidad tiene que ser lento: para frenar, el robot primero acelera las ruedas (fase no mínima), y con ganancias altas se vuelve inestable.

4.5. Pipeline común

Los cuatro algoritmos pasan por los mismos scripts de entrenamiento, afinamiento y evaluación (`training/train.py`, `training/tune.py`, `evaluation/`), y solo cambia el archivo YAML de cada uno. La red es fija para todos: una MLP $[256, 256]$. Cada corrida guarda la semilla, la configuración completa, las versiones de las librerías, el commit de git, la máquina y la duración. Ningún hiperparámetro ni constante física está escrito en el código: todo sale de `configs/` o del MJCF. El repositorio tiene 112 pruebas automáticas. La tabla 4 resume su organización.

Tabla 4: Organización del repositorio.

Carpeta	Contenido
`assets/`	Modelo MJCF del robot (`balance_bot.xml`).
`envs/`	Entorno `BalanceBotEnv`, comandos, empujones, aleatorización de dominio y grabación de video.
`baselines/`	PID en cascada.
`configs/`	YAML de niveles, algoritmos, entrenamiento, afinamiento, PID y evaluación.
`training/`	Entrenamiento, afinamiento con Optuna y *callbacks* de evaluación y diagnóstico.
`evaluation/`	Evaluación en los niveles 0–3 y prueba de empujón.
`analysis/`	Estadística entre semillas, figuras, tablas y videos finales.
`lab/`	Herramientas para afinar y correr semillas en otras máquinas y en Google Colab.
`tests/`	Pruebas automáticas con `pytest`.
`docs/`	Especificación, entregables, resultados en Markdown y este informe.

4.6. Afinamiento

Cada algoritmo se afinó con Optuna en el nivel 2, con el mismo presupuesto: 40 pruebas de 200 000 pasos, muestreador TPE y poda por mediana. El objetivo es la media de las últimas tres evaluaciones. Se afinaron los hiperparámetros que la literatura señala como críticos para cada algoritmo (tabla 5).

Tabla 5: Mejor configuración encontrada por algoritmo (40 pruebas × 200 000 pasos, nivel 2).

Algoritmo	Objetivo	Hiperparámetros elegidos
DDPG	836.3	lr 8.85×10^{-4}, ruido de exploración $\sigma = 0.236$, $\tau = 0.0171$, batch 256
TD3	896.9	lr 1.95×10^{-3}, ruido de la política objetivo 0.448, retraso de la política 4, $\sigma = 0.478$
SAC	878.1	lr 2.29×10^{-4}, coeficiente de entropía automático, $\tau = 0.0055$, batch 128
PPO	865.9	lr 1.61×10^{-4}, clip 0.163, 16 épocas, 2048 pasos por rollout, $\lambda = 0.950$

4.7. Protocolo experimental

Con la mejor configuración, cada algoritmo se entrenó un millón de pasos en el nivel 2 con 10 semillas, y en el nivel 3 con 5 semillas (experimento E5). La evaluación final usa la política determinista, 50 episodios por nivel con semillas fijas, distintas de las de entrenamiento e iguales para todas las políticas. Entre semillas se reporta el IQM (media intercuartil) con intervalos de confianza al 95 % por bootstrap.

Las 60 corridas finales se repartieron entre tres tipos de máquina: la laptop de Mario (DDPG y PPO), Google Colab (tres semillas de DDPG) y la computadora de Christian (TD3 y SAC). Como el desempeño no depende de la máquina pero el tiempo de reloj sí, E2 solo compara algoritmos que corrieron en la misma máquina.

5. Herramientas aplicadas

Se detallan con más cuidado las herramientas que no se vieron en clase: MuJoCo, Optuna, rliable y la herramienta de IA generativa.

5.1. MuJoCo

MuJoCo (*Multi-Joint dynamics with Contact*) [17] es un motor de física para robótica, de código abierto desde 2022, con *bindings* oficiales de Python (paquete `mujoco`). Se usa en investigación de RL porque simula contactos de forma rápida y estable.

- **MJCF.** El robot se describe en un archivo XML (`assets/balance_bot.xml`) con cuerpos, articulaciones, geometrías, actuadores y sensores. El chasis es un cuerpo libre; cada rueda es un cuerpo hijo con una articulación de bisagra y un actuador de torque limitado a $\pm 0.3\,\mathrm{N\,m}$.
- **Sensores.** MuJoCo calcula sensores como lo haría el hardware: `framequat` (orientación de la IMU), `gyro` (velocidad angular, con corte en $34.9\,\mathrm{rad/s} \approx 2000\,^\circ/\mathrm{s}$), `accelerometer` y `jointvel` en cada rueda (encoders). La observación del entorno sale solo de estos sensores.
- **Integración.** Paso de 2 ms (500 Hz) con el integrador `implicitfast`, más estable que Euler para el amortiguamiento de las ruedas; el entorno aplica cada acción durante 5 pasos de física (control a 100 Hz).
- **Perturbaciones y aleatorización.** Los empujones se aplican como una fuerza externa sobre el chasis (`xfrc_applied`); la aleatorización del nivel 3 modifica la masa e inercia del

chasis, la fricción de las ruedas y la relación de transmisión (`gear`) de los actuadores en el modelo al inicio de cada episodio.

- **Render.** El renderizado sin ventana (`mujoco.Renderer`) produce los cuadros de los videos.

5.2. Gymnasium

Gymnasium [18] define la interfaz estándar de un entorno de RL: `reset()` y `step(a)`, que devuelve observación, recompensa y las banderas `terminated` (el robot cayó) y `truncated` (se acabó el tiempo). Distinguirlas es importante: al truncar, el valor del estado siguiente no es cero. El entorno pasa el verificador `check_env` de Gymnasium.

5.3. Stable-Baselines3

Stable-Baselines3 [12] ofrece implementaciones probadas de DDPG, TD3, SAC y PPO con la misma interfaz, lo que permite que el pipeline sea uno solo. Se usaron sus *callbacks* para evaluar cada 10 000 pasos y para los diagnósticos internos, y su registro en TensorBoard. Los algoritmos no se implementaron desde cero: el aporte del proyecto es el entorno, el protocolo y el análisis.

5.4. Optuna

Optuna [2] es una librería de optimización de hiperparámetros. Un *estudio* contiene *pruebas*; en cada prueba, una función objetivo pide valores (`trial.suggest_float`, `suggest_categorical`), entrena y devuelve un número que Optuna maximiza.

- **Muestreador TPE** [4]. El *Tree-structured Parzen Estimator* separa las pruebas ya hechas en buenas y malas, modela la densidad de los hiperparámetros en cada grupo y propone los valores que maximizan el cociente entre ambas densidades. Las primeras pruebas son aleatorias, para tener con qué modelar.
- **Poda por mediana.** Durante el entrenamiento, cada prueba reporta su retorno de evaluación. Si en un mismo punto queda por debajo de la mediana de las pruebas anteriores, se detiene. Esto ahorra cómputo en configuraciones que claramente no funcionan.
- **Almacenamiento en SQLite.** El estudio vive en un archivo `.db`, así que varios procesos pueden afinar a la vez el mismo estudio y un afinamiento cortado se puede reanudar. Para Colab y otras máquinas, `lab/` empaqueta y luego une los estudios.
- **Objetivo.** La media de las últimas tres evaluaciones, para no premiar un pico aislado.

5.5. rliable

rliable [1] implementa la metodología propuesta por Agarwal et al. para reportar resultados de RL con pocas semillas:

- **IQM (media intercuartil).** Promedio del 50 % central de las corridas: descarta el 25 % más bajo y el 25 % más alto. Es más robusto que la media ante una semilla atípica y más eficiente que la mediana.
- **Intervalos de confianza por bootstrap.** Se remuestrean las semillas con reemplazo muchas veces, se calcula el IQM de cada remuestra y los percentiles 2.5 y 97.5 dan el intervalo al 95 %.

El módulo `rliable.library` no funciona con pandas 3 en este entorno, así que el IQM sale de `rliable.metrics` y el bootstrap se implementó aparte con el mismo método.

5.6. Otras herramientas

- **PyTorch**, como motor de las redes de SB3 (solo CPU).
- **TensorBoard**, para seguir las curvas y los diagnósticos durante el entrenamiento.
- **NumPy, pandas y Matplotlib**, para el análisis, las tablas y las figuras.
- **imageio con FFmpeg**, para los videos de los controladores.
- **pytest**, con 112 pruebas del entorno, el PID, el pipeline y la evaluación.
- **Git y GitHub**, para el código y la documentación; las corridas finales se versionaron para compartirlas entre máquinas.
- **Google Colab**, para correr semillas y pruebas de afinamiento adicionales.

5.7. Herramienta de IA generativa

Usamos Claude, de Anthropic, en dos etapas y de dos formas distintas:

1. **Definición del proyecto** (15 de octubre al 1 de noviembre): Claude en conversación [5], como interlocutor técnico para escoger el tema, delimitar el alcance y escribir la documentación base del repositorio.
2. **Implementación** (1 al 9 de noviembre): Claude Code [6], el agente de programación que trabaja en la terminal sobre el repositorio, guiado por esa documentación.

El registro completo de la primera etapa está en `docs/uso-ia.md`.

5.7.1. Etapa 1: definición del proyecto

Fue un proceso de ida y vuelta: nosotros planteábamos ideas y restricciones, Claude proponía opciones y señalaba riesgos, y nosotros decidíamos. La tabla 6 resume la conversación; en esta etapa los prompts están resumidos, no textuales.

Tabla 6: Definición del proyecto con Claude: lo que planteamos, lo que propuso y lo que decidimos.

Lo que planteamos	Lo que propuso Claude	Lo que decidimos
Un proyecto basado en lo último visto en clase (Deep Q-Learning), con dificultad acorde a un equipo de cinco.	Cinco opciones: semáforos multiagente en SUMO, ablación de Rainbow DQN, *self-play* en un juego de tablero, ajuste de un LLM con GRPO y manejo autónomo en `highway-env`. Desaconsejó el *trading* con RL por el sobreajuste al histórico.	—
El equipo quedó en dos personas: ¿qué sigue siendo viable?	Redimensionó las opciones, descartó la del LLM por requerir GPU y sugirió otras: *autoscaling*, inventario, batería solar, MiniGrid y *reward hacking*.	—
Un tema propio: robot de auto-balanceo en MuJoCo comparando DDPG, TD3, SAC y PPO.	Lo consideró adecuado, pero señaló riesgos: un PID resuelve bien el balanceo puro, faltaba una línea base clásica, el entorno había que construirlo y la comparación necesitaba varias semillas.	El tema es nuestro. Por los riesgos, agregamos niveles de dificultad crecientes y un PID/LQR de referencia.
¿Podemos usar implementaciones existentes de los algoritmos?	Advirtió el riesgo de "correr `model.learn()` cuatro veces" y propuso evitarlo con el mismo presupuesto de afinamiento en Optuna, el esfuerzo puesto en el entorno y diagnósticos internos.	Usar Stable-Baselines3 con ese protocolo.
Un documento inicial en Markdown; ¿es viable un robot físico?	Redactó la especificación con valores marcados como propuesta y decisiones abiertas. Explicó los problemas de pasar de simulación a la realidad y luego propuso ampliar el alcance al robot físico.	Quedarnos en simulación, sin cerrar la puerta al robot real.
La documentación para el repositorio, orientada a agentes de código.	Redactó `README.md`, `AGENTS.md`, `docs/fase-1-entorno.md`, `requirements.txt` y `.gitignore`.	Adoptar la estructura y las reglas de trabajo.

En resumen, el tema, el uso de librerías existentes y la decisión de no construir un robot físico fueron nuestros. Claude aportó sobre todo la evaluación de riesgos, la estructura del protocolo experimental, el diseño de los cuatro niveles y la redacción de los documentos para agentes de código.

5.7.2. Etapa 2: implementación

Los dos integrantes usamos Claude Code a partir de la misma documentación, que funciona como contrato entre el equipo y el agente:

- `docs/especificacion-proyecto.md`: la fuente de verdad del proyecto. Define la pregunta, el entorno, la recompensa, los algoritmos, el protocolo, los diagnósticos y una lista de *decisiones abiertas* (sección 13) que solo el equipo puede cerrar. Todo cambio se anota en una bitácora (sección 14).
- `AGENTS.md`: las reglas para el agente. Analizar antes de programar, proponer antes de modificar un archivo existente, no decidir solo las decisiones abiertas, no escribir hiperparámetros en el código, usar solo observaciones de IMU y encoders, una prueba por componente y correr `pytest` antes de terminar.

- `docs/entregables.md`: el plan por entregables (E0–E13), con responsable y estado.

El agente no decidía el diseño: proponía, el equipo aprobaba o corregía, y las decisiones abiertas (por ejemplo PID o LQR, el número de semillas o la normalización de observaciones) se cerraron entre los dos y quedaron registradas.

La tabla 7 reúne prompts de las sesiones de Christian, en orden cronológico y textuales (solo se recortaron con [...]). La columna *Resultado* es un resumen nuestro. Muchos prompts fueron solo aprobaciones cortas, como "VA sigamos con todo", "continua porfa" o "Va vamos con ello": el plan estaba en la especificación y en los entregables, así que el agente proponía el siguiente paso y nosotros lo aprobábamos o corregíamos. Mario trabajó de la misma forma, con la misma documentación; un resumen de una de sus sesiones está en `docs/resumen-sesion-2026-11-07.md`.

Tabla 7: Prompts de la implementación con Claude Code y lo que produjeron.

Fecha	Prompt	Resultado
1 nov	"Como estas, mira por favor revisate los documentos dentro de este repositorio por favor, [...] hagamos por medio de entregables todo, [...] estoy trabajando con un compañero en este proyecto por lo que el hará la implementacion de DDPG y PPO y yo lo demas"	Revisión de la especificación, plan por entregables (`docs/entregables.md`) y reparto por algoritmo.
1 nov	"1. Vamos con las correcciones [...] 2. Si usemos SB3, mi compañero igual lo utilizará. 3. La fecha de entrega es por ahi del 13 de noviembre, pero hay que hacer un entregable para el 2"	Correcciones a la especificación y calendario con el avance del 2 de noviembre.
1 nov	"Anota los valores encontrados por favor creo que si es pertinente hacerlo"	Resultados de las pruebas del modelo físico (fricción, amortiguamiento, integrador, saturación de sensores) anotados en la especificación.
1 nov	"Va vamos con eso, veamos lo de los nucleos y que pasa con el tiempo estimado" / "estos tiempos que tanto crees que se acortarian si es que lo hicieramos en algo como google colab?"	Medición de pasos por segundo y estimación del cómputo; origen de las herramientas para correr en varias máquinas.
2 nov	"VA sigamos con todo" / "continua porfa" / "Va vamos con ello"	Aprobaciones para seguir con el plan de la fase 2.
2 nov	"Hazte otro porfa dejemos todo listo para el avance de hoy" / "Te podrias crear un documento latex con los avances para la entrega"	Pipeline común, niveles 1–3, prueba de humo de TD3 y SAC y documento LaTeX del avance.
9 nov	"revisa lo que acabo de pullear porfavor y dame update, mi compañero tambien me envio esto: [estado de las corridas de DDPG y PPO]"	Integración del trabajo de Mario; plan para afinar TD3 y SAC (40×200k) y correr sus semillas; confirmación de 10 y 5 semillas.
9 nov	"olvide mi cargador y estoy cargando la computadora con un cargador de solo 65 watts, [...] crees que podamos correr esto con la extension de colab de vscode?"	Pausa de los procesos, comparación con Colab y, después, un guardia de batería que pausaba el entrenamiento si la carga bajaba.
9 nov	"Documentos de seguimiento: [...] termina esto tambien, solo me queda duda con los videos si solo con esos dos esta"	Especificación y entregables al día y script de videos de los controladores finales.
9 nov	"Me gusta un monton la presentacion, pero quitemos toda la IA slop, subtitulos, puntos para separacion de titulos [...] intentemos con otro tipo de brief"	Rediseño de la presentación.
9 nov	[Requisitos del documento final] "crees que cumplimos con todo?"	Revisión contra la rúbrica y reorganización de este informe.

5.7.3. Evolución de la idea

1. **Elección del tema (desde el 15 de octubre).** Partimos de una lluvia de ideas para un equipo de cinco basada en Deep Q-Learning. Al quedar en dos, escogimos un tema propio: comparar algoritmos de control continuo en un robot de auto-balanceo. La discusión de riesgos con Claude convirtió una comparación simple en un estudio con niveles de dificultad, un PID de referencia, el mismo presupuesto de afinamiento y diagnósticos internos. Descartamos el robot físico.
2. **Especificación y documentación (19 de octubre al 1 de noviembre).** La especifi-

cación nace con la pregunta de investigación, los cuatro algoritmos, una línea base "PID o LQR", afinamiento con Optuna y niveles de dificultad, con varios valores marcados como propuesta o decisión abierta. Junto con ella se escriben `AGENTS.md` y el plan de la fase 1.

3. **Fase 1, entorno (1–2 de noviembre).** Se construyen el MJCF, el entorno del nivel 0 y el PID. Las pruebas del modelo fijan la fricción, el amortiguamiento, el integrador y los límites de los sensores, que se anotan en la especificación. Se elige el PID en cascada como línea base porque balancea 20 de 20 episodios en el nivel 0. Aparece el problema de la fase no mínima: el lazo de velocidad tiene que ser lento.

4. **Fase 2, pipeline (2 de noviembre).** Se escriben `train.py` y los niveles 1–3, y una prueba de humo confirma que los cuatro algoritmos aprenden con el mismo código. Con esto se entrega el avance.

5. **Fase 3, afinamiento y diagnósticos (5–7 de noviembre, Mario).** Configuraciones de DDPG y PPO, `tune.py` con Optuna, diagnósticos de sobreestimación de Q y suavidad, y las herramientas de `lab/` para afinar y correr semillas en otras computadoras y en Colab, porque el cómputo de una sola laptop no alcanzaba.

6. **Corridas finales (7–9 de noviembre).** DDPG y PPO corren en la laptop de Mario y en Colab; TD3 y SAC se afinan y corren en la computadora de Christian. Las limitaciones prácticas (horas de cómputo, un cargador insuficiente) llevan a fijar 10 semillas en el nivel 2 y 5 en el nivel 3, y a no normalizar observaciones.

7. **Evaluación y análisis (9 de noviembre).** Se escriben la evaluación en los cuatro niveles y la prueba de empujón. La primera versión medía 3 s tras el empujón y el tiempo de recuperación del PID salía indefinido; al revisar se vio que el PID tarda unos 3 s en recuperarse y la ventana se amplió a 6 s. Después vienen el análisis estadístico, este informe, los videos y la presentación.

5.7.4. Entendimiento de la solución

La IA aceleró la escritura de código y documentos, pero el equipo mantuvo el control y el entendimiento de la solución de varias formas:

- **Las decisiones fueron del equipo.** Las reglas de `AGENTS.md` obligan al agente a detenerse ante una decisión abierta y a proponer antes de modificar un archivo existente. Cada decisión cerrada quedó en la bitácora de la especificación con su autor.
- **Las hipótesis se escribieron antes de los resultados**, y se rechazan cuando los datos lo indican (H2) en lugar de ajustarlas.
- **Cada componente tiene pruebas**, y `pytest` se corre antes de dar una tarea por terminada.
- **Los resultados se explican por los mecanismos** de cada algoritmo: el mínimo de dos críticos de TD3 contra la sobreestimación de DDPG, las acciones suaves de PPO frente al ruido, la política conservadora que produce la aleatorización. Esos argumentos se verificaron con los diagnósticos internos, no solo con el retorno.
- **Los errores se detectaron revisando los datos**, como la ventana de la prueba de empujón o las semillas de DDPG en Colab, que se excluyeron de la comparación de tiempo de reloj.

6. Resultados

6.1. Métricas

- **Retorno**: suma de la recompensa del episodio; el máximo es 1000. El *retorno final* de una corrida es la media de sus últimas tres evaluaciones durante el entrenamiento.
- **Pasos y horas al 90 %**: pasos de entorno y tiempo de reloj hasta alcanzar el 90 % del retorno final.
- **Tasa de caídas**: fracción de episodios que terminan porque $|\theta| > 0.6\,\text{rad}$.
- **RMSE de seguimiento**: error cuadrático medio entre la velocidad lineal o de giro y su comando.
- **Suavidad**: $\|a_t - a_{t-1}\|$ promedio. **Esfuerzo**: $\|a_t\|^2$ promedio. **Energía de alta frecuencia**: fracción de la energía de la acción por encima de 5 Hz.
- **Fuerza máxima tolerada**: la mayor fuerza de un empujón tras la cual el robot no cae, buscada por bisección.
- **Tiempo de recuperación**: tiempo desde el empujón hasta que $|\theta| < 2\check{r}$ de forma sostenida durante 0.5 s.
- **Sobreestimación de Q**: diferencia entre $Q(s, \pi(s))$ del crítico y el retorno descontado real obtenido desde esos estados.

6.2. E1 y E2: eficiencia de muestras y de tiempo

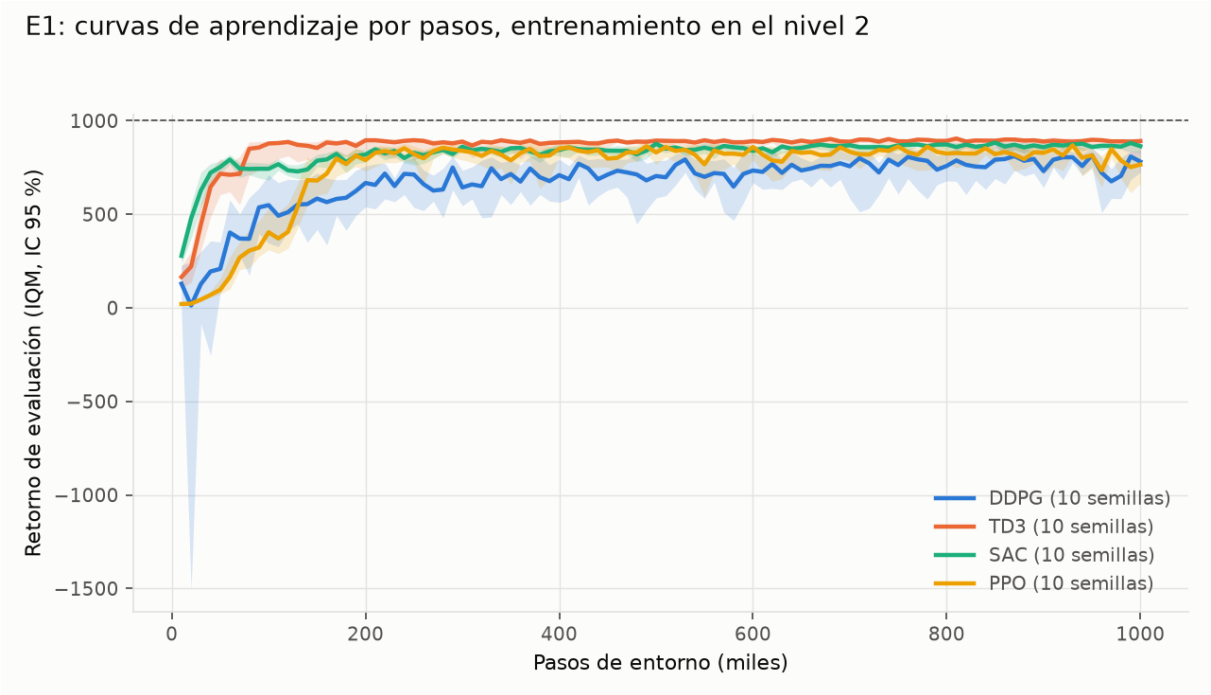


Figura 1: Retorno de evaluación durante el entrenamiento en el nivel 2 (IQM de 10 semillas e IC 95 %).

Tabla 8: Desempeño al final del entrenamiento en el nivel 2 (10 semillas). Retorno final: media de las últimas 3 evaluaciones de cada corrida. Las horas se miden en la máquina donde corrió cada algoritmo (laptop i7 de 4 núcleos o Ryzen 7 de 8; las 3 semillas de DDPG en Colab no cuentan).

Algoritmo	IQM del retorno final	Peor / mejor	Pasos al 90 %	Horas al 90 %	Máquina
DDPG	756.5 [674.5, 809.6]	511 / 871	175k	1.43	laptop
TD3	**891.1** [832.7, 897.8]	572 / 907	75k	0.27	Ryzen 7
SAC	861.3 [844.7, 876.2]	**821** / 885	**50k**	0.43	Ryzen 7
PPO	767.1 [678.5, 813.2]	410 / 856	145k	0.28	laptop

- **TD3 llega al mejor desempeño** y se mantiene estable durante todo el entrenamiento (figura 1).
- **SAC es el más eficiente en muestras** (90 % de su retorno final en 50 000 pasos) y el más consistente: su peor semilla da 821 y su intervalo es el más angosto.
- **DDPG es el más lento e inestable**, con caídas bruscas en las curvas y la peor semilla en 511.
- **E2:** el tiempo de reloj solo se compara dentro de una misma máquina. En la laptop, PPO llega al 90 % en 0.28 h y DDPG en 1.43 h: aunque PPO necesita más pasos que TD3 o SAC, cada paso le cuesta unas 4 veces menos, porque hace muchas menos actualizaciones de gradiente por paso de entorno. En el Ryzen, TD3 (0.27 h) es más rápido que SAC (0.43 h). DDPG 4–6 corrieron en Colab y no entran en esta comparación.

6.3. E3 y E6: generalización a los niveles 0–3 y comparación con el PID

Tabla 9: Retorno y tasa de caídas al evaluar en cada nivel a los agentes entrenados en el nivel 2 (IQM de 10 semillas, 50 episodios por nivel). El PID es determinista: se reporta su media.

Nivel		DDPG	TD3	SAC	PPO	PID
0	Retorno	998	999	**1000**	994	999
1	Retorno	947	975	958	924	**995**
2	Retorno	748	**891**	854	769	807
3	Retorno	283	374	257	**537**	−471
1	Caídas	5 %	1 %	4 %	14 %	**0 %**
2	Caídas	8 %	**0 %**	2 %	16 %	**0 %**
3	Caídas	51 %	**36 %**	60 %	39 %	72 %

Tabla 10: Seguimiento de comandos y suavidad de las acciones, agentes entrenados en el nivel 2 (IQM).

	DDPG	TD3	SAC	PPO	PID
RMSE v, nivel 2 (m/s)	0.496	**0.350**	0.404	0.479	0.545
RMSE v, nivel 3 (m/s)	0.632	**0.550**	0.667	0.580	1.522
RMSE $\dot{\psi}$, nivel 2 (rad/s)	0.459	0.226	0.232	0.310	**0.123**
RMSE $\dot{\psi}$, nivel 3 (rad/s)	1.281	0.891	1.134	**0.475**	2.954
$\|a_t - a_{t-1}\|$, nivel 3	0.472	0.464	0.488	**0.183**	0.314
Esfuerzo $\|a\|^2$, nivel 3	0.457	0.567	0.704	**0.135**	1.180

- En el nivel de entrenamiento (2), **TD3 y SAC superan al PID**; DDPG y PPO quedan ligeramente por debajo, aunque siguen mejor la velocidad que el PID.

- En los niveles 0 y 1, sin comandos, el PID es tan bueno o mejor que los agentes: es el escenario para el que se ajustó.
- **En el nivel 3, todos los agentes superan con claridad al PID**, que cae en el 72 % de los episodios. Ningún agente vio ruido ni latencia durante su entrenamiento.
- **PPO es el que mejor generaliza al nivel 3**: el mejor retorno, el mejor seguimiento del giro y acciones mucho más suaves (0.18 contra ∼0.47 de los métodos off-policy) y con menos esfuerzo. Los algoritmos off-policy aprenden políticas deterministas muy reactivas, que amplifican el ruido de los sensores; es el mismo mecanismo que hace fallar al PID, con ganancias altas en el ángulo.

6.4. E4: prueba de empujón

En el nivel 0 se aplica un único empujón a $t = 2\,s$ en la parte alta del chasis, hacia adelante y hacia atrás, desde cinco estados iniciales. La fuerza máxima tolerada se busca por bisección.

Tabla 11: Prueba de empujón, agentes entrenados en el nivel 2 (IQM de 10 semillas e IC 95 %).

Política	Fuerza máxima (N)	Recuperación a 4 N (s)	Recuperación a 8 N (s)
DDPG	10.5 [9.9, 12.0]	0.96 [0.87, 1.29]	1.45 [1.18, 1.72]
TD3	12.7 [11.2, 13.6]	**0.78** [0.69, 0.92]	**1.06** [0.98, 1.20]
SAC	12.2 [11.7, 12.9]	1.10 [0.91, 1.26]	1.38 [1.27, 1.57]
PPO	9.0 [8.5, 10.0]	1.30 [1.16, 1.40]	1.82 [1.70, 2.37]
PID	**17.2**	2.58	3.35

El PID tolera la mayor fuerza aislada, pero tarda unas tres veces más en volver a la vertical, porque frena inclinándose de forma lenta y controlada. Los agentes recuperan la vertical en alrededor de un segundo, a cambio de un margen menor ante empujones extremos. TD3 es el mejor de los agentes en las dos métricas.

6.5. E5: entrenar con aleatorización de dominio

Tabla 12: Efecto de entrenar en el nivel 3 (5 semillas por algoritmo) frente a entrenar en el nivel 2. Retorno IQM al evaluar en cada nivel; entre paréntesis, el valor del agente entrenado en el nivel 2.

Evaluado en	DDPG	TD3	SAC	PPO
Nivel 1	958 (947)	878 (975)	898 (958)	691 (924)
Nivel 2	794 (748)	718 (891)	673 (854)	513 (769)
Nivel 3	**654** (283)	623 (374)	649 (257)	479 (537)
Caídas en el nivel 3	**19 %** (51 %)	41 % (36 %)	41 % (60 %)	69 % (39 %)
Fuerza máxima (N)	11.8 (10.5)	8.8 (12.7)	9.8 (12.2)	6.6 (9.0)

- Entrenar con aleatorización **sube el retorno en el nivel 3 para DDPG, TD3 y SAC** (por ejemplo, DDPG pasa de 283 a 654 y de 51 % a 19 % de caídas).
- A cambio, la mayoría **empeora en los niveles sin ruido** y tolera empujones menores: aprende una política más conservadora. DDPG es la excepción: mejora en los niveles 1 a 3 y queda igual en el 0.
- **PPO no se beneficia**: entrenado en el nivel 3 rinde peor en todos los niveles, incluso en el 3. Con el mismo presupuesto de pasos, el ruido en las observaciones parece dificultar su estimación de ventajas on-policy.

- Con 5 semillas los intervalos son anchos (por ejemplo, TD3 en el nivel 3: [415, 682]), así que estas diferencias deben leerse como tendencias.

6.6. Diagnósticos internos

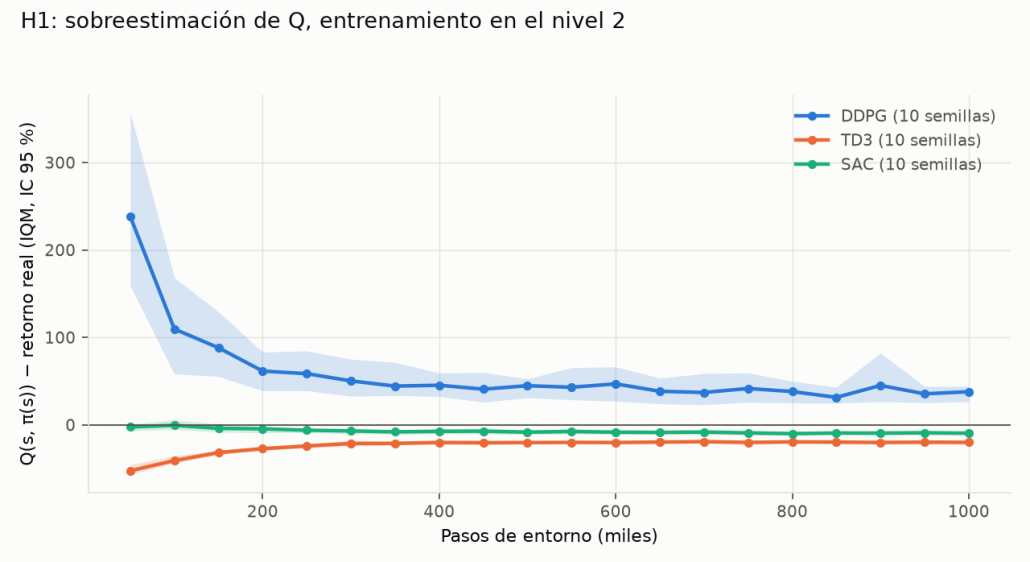


Figura 2: Sobreestimación de Q: diferencia entre $Q(s, \pi(s))$ del crítico y el retorno descontado real obtenido desde esos estados, cada 50 000 pasos (IQM de 10 semillas).

Tabla 13: Diagnósticos al final del entrenamiento en el nivel 2 (media de los 3 últimos diagnósticos, IQM entre semillas).

Algoritmo	Sobreestimación de Q	$\|a_t - a_{t-1}\|$	Energía de la acción > 5 Hz
DDPG	+41.0 [28.3, 56.2]	0.110	0.49
TD3	−19.9 [−22.4, −18.1]	0.038	0.37
SAC	−9.1 [−11.9, −7.2]	0.041	0.44
PPO	—	0.039	**0.22**

DDPG empieza sobreestimando Q en más de 200 puntos y termina en +41 (figura 2). TD3, con el mínimo de dos críticos, subestima de forma estable (−20), y SAC, que también usa doble crítico pero suma el bono de entropía, queda cerca de cero. La sobreestimación de DDPG coincide con su entrenamiento más inestable y con acciones casi tres veces más bruscas.

6.7. Contraste de hipótesis

H1 — **se confirma.** DDPG sobreestima Q (+41) y TD3 no (−20), con intervalos que no se traslapan. TD3 entrena de forma más estable (peor semilla 572 contra 511, curvas sin caídas) y con acciones más suaves.

H2 — **se rechaza.** SAC no se recupera mejor que TD3: tolera una fuerza similar (12.2 contra 12.7 N) y tarda más en recuperarse (1.38 contra 1.06 s a 8 N). La entropía le da consistencia entre semillas y eficiencia de muestras, pero no una mejor respuesta a los empujones.

H3 — **se confirma solo en parte.** PPO necesita más pasos que TD3 y SAC (145k contra 75k y 50k), pero menos que DDPG, y su varianza entre semillas no es menor: tiene la peor semilla (410) y uno de los intervalos más anchos. En tiempo de reloj, en cambio, es el más rápido.

H4 — se confirma con matices. Los cuatro agentes igualan al PID en el nivel 0 y lo superan con claridad en el nivel 3. En el nivel 2, solo TD3 y SAC lo superan en retorno, aunque los cuatro siguen mejor la velocidad. El PID conserva dos ventajas: no se cae en los niveles 1 y 2, y tolera el empujón aislado más fuerte.

7. Conclusión

TD3 es el mejor algoritmo para esta tarea en las condiciones para las que se entrena: tiene el mayor retorno, la menor tasa de caídas, el mejor seguimiento de velocidad y la recuperación más rápida ante empujones. El mecanismo que lo explica es el que predice H1: el doble crítico con el mínimo y el retraso de la política eliminan la sobreestimación de Q que desestabiliza a DDPG.

SAC es la alternativa más confiable: queda cerca de TD3, aprende con menos muestras y casi no depende de la semilla.

Si el robot va a operar con sensores ruidosos y latencia sin haberse entrenado para eso, PPO es la mejor opción: produce acciones más suaves y con menos esfuerzo, que no amplifican el ruido. **Si se puede entrenar con aleatorización de dominio**, DDPG, TD3 y SAC recuperan esa robustez, y DDPG es el que más se beneficia.

Frente al control clásico, el aprendizaje por refuerzo aporta sobre todo robustez ante ruido y latencia, y mejor seguimiento de comandos. El PID sigue siendo competitivo en condiciones limpias y tolera mejor los empujones aislados muy fuertes.

Limitaciones

- El afinamiento usa una sola semilla por prueba, así que la mejor configuración puede deberse en parte al azar. No hubo tiempo de revalidarla con más semillas.
- El nivel 3 tiene 5 semillas por algoritmo; sus intervalos son anchos.
- Las corridas se repartieron entre tres tipos de máquina (laptop, Ryzen 7 y Colab). El desempeño no depende de la máquina, pero el tiempo de reloj (E2) solo se compara dentro de cada una. DDPG 4–6 corrieron en Colab con Python 3.13; las demás librerías son iguales.
- No se normalizaron las observaciones: la prueba inicial con una semilla no fue concluyente y la decisión se aplicó igual a los cuatro algoritmos.
- El PID se ajustó en el nivel 0 y no se reajustó para el ruido del nivel 3. Un PID con filtros o ganancias más bajas probablemente reduciría su desventaja ahí.

Código y video

Todo el código, las configuraciones, los resultados de las corridas finales y este informe están en el repositorio público `https://github.com/Ctribsz/ProyectoFinal-APR`. El video que muestra el funcionamiento de la solución, su implementación y los principales hallazgos está en `https://youtu.be/TeNxtZMx2g0`.

Reparto del trabajo

- **Christian Echeverría:** modelo físico, entorno y niveles, PID, pipeline de entrenamiento, evaluación y prueba de empujón, TD3 y SAC (afinamiento y corridas finales).
- **Mario Betancourt:** DDPG y PPO (configuración, espacio de búsqueda, afinamiento y corridas finales), afinamiento con Optuna, diagnósticos internos, herramientas para correr en otras máquinas y en Colab, y el análisis estadístico de las corridas finales.
- **Ambos:** análisis, informe y dictamen.

Referencias

[1] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville y M. G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. Optuna: A next-generation hyperparameter optimization framework. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019.

[3] K. J. Åström y R. M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.

[4] J. Bergstra, R. Bardenet, Y. Bengio y B. Kégl. Algorithms for hyper-parameter optimization. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.

[5] Anthropic. Claude. `https://claude.ai`. Consultado en octubre y noviembre de 2026.

[6] Anthropic. Claude Code. `https://www.anthropic.com/claude-code`. Consultado en noviembre de 2026.

[7] S. Fujimoto, H. van Hoof y D. Meger. Addressing function approximation error in actor-critic methods. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[8] T. Haarnoja, A. Zhou, P. Abbeel y S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[9] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel y S. Levine. Soft actor-critic algorithms and applications. *arXiv:1812.05905*, 2018.

[10] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup y D. Meger. Deep reinforcement learning that matters. En *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[11] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver y D. Wierstra. Continuous control with deep reinforcement learning. En *International Conference on Learning Representations (ICLR)*, 2016.

[12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus y N. Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[13] J. Schulman, P. Moritz, S. Levine, M. Jordan y P. Abbeel. High-dimensional continuous control using generalized advantage estimation. En *International Conference on Learning Representations (ICLR)*, 2016.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford y O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[15] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*. 2.ª ed., MIT Press, 2018.

[16] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba y P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. En *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[17] E. Todorov, T. Erez y Y. Tassa. MuJoCo: A physics engine for model-based control. En *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[18] M. Towers et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv:2407.17032*, 2024.

[19] H. van Hasselt. Double Q-learning. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2010.